

From Click to Code: Instantly Generate Your UI Test Scripts

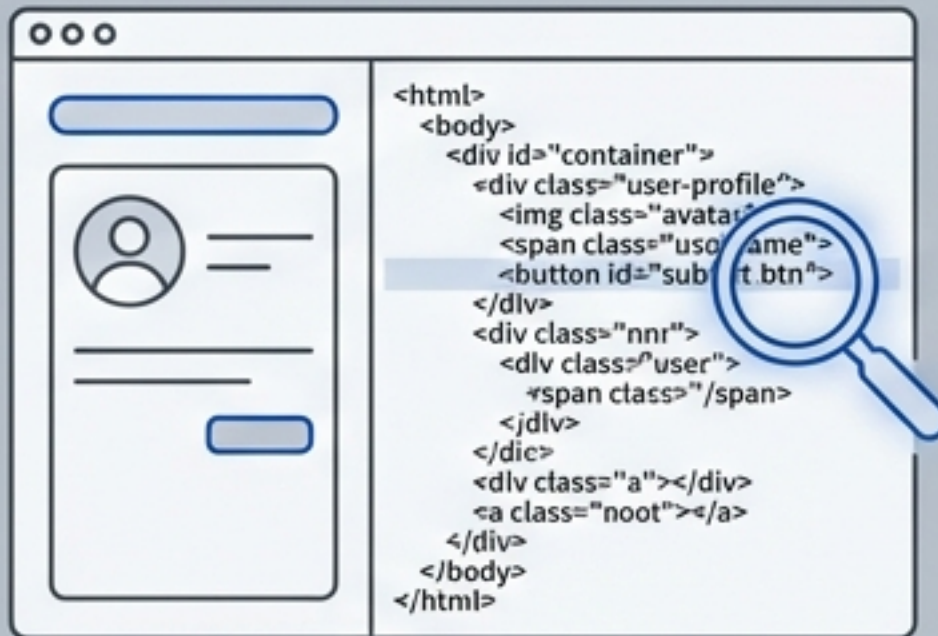
An architectural and functional overview of an AI-powered browser extension that accelerates test automation.



The Manual Grind of UI Test Automation

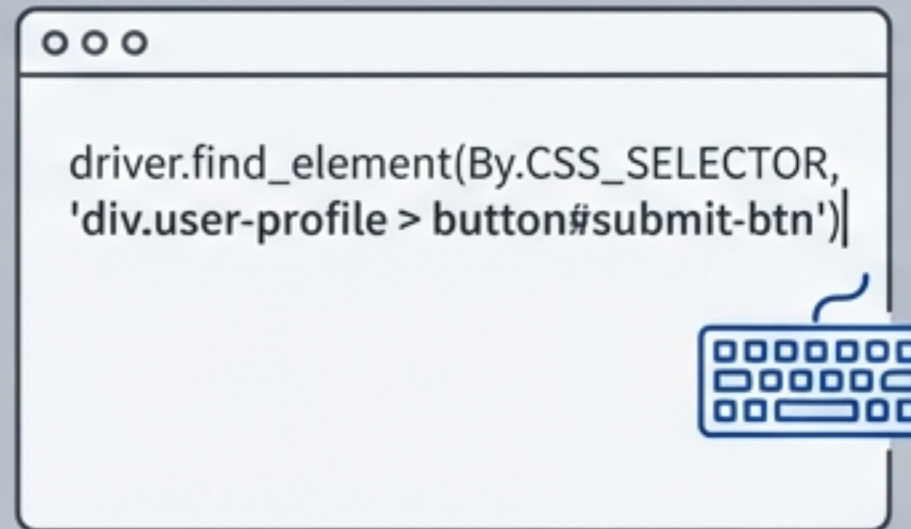
Key idea: Writing selectors for UI tests is repetitive, time-consuming, and error-prone.

Inspect Element



Manually inspecting the DOM to find the right element.

Write Selector



Crafting and debugging brittle selectors.

Repeat



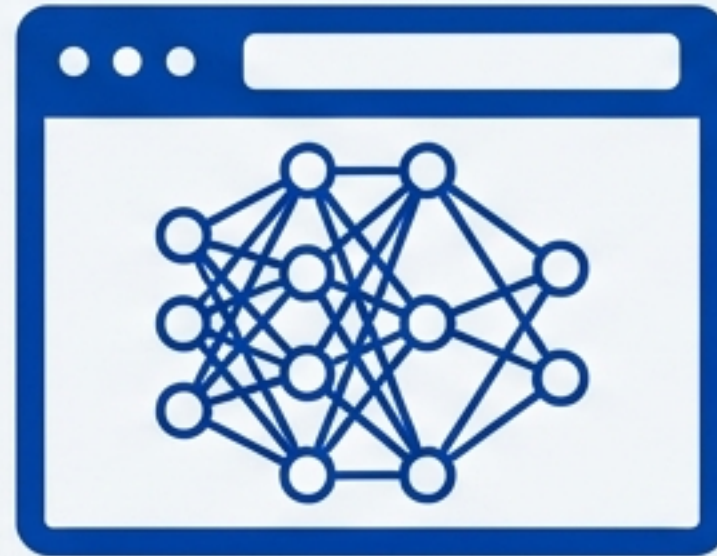
Repeating the process for every single element in a test flow.

A Smarter Way: The AI-Powered Test Script Generator

An intelligent browser extension that eliminates manual selector creation by generating production-ready test code with a single click.



Instant Generation: Get Selenium/Playwright code in 2-5 seconds.



Framework-Ready: Directly copy and paste into your test automation framework.



Simple Workflow: Inspect, click, and copy.

The Five-Step User Workflow



1

SETUP (One-Time)

Install the extension and enter your Groq or OpenAI API key.



2

INSPECT (User Action)

Open a webpage and click the 'Inspect' button in the extension.



3

GENERATE (LLM Magic)

Click on a page element; the extension sends it to the LLM for code generation.



4

DISPLAY (See the Code)

Generated Selenium/Playwright code appears instantly in the extension panel.

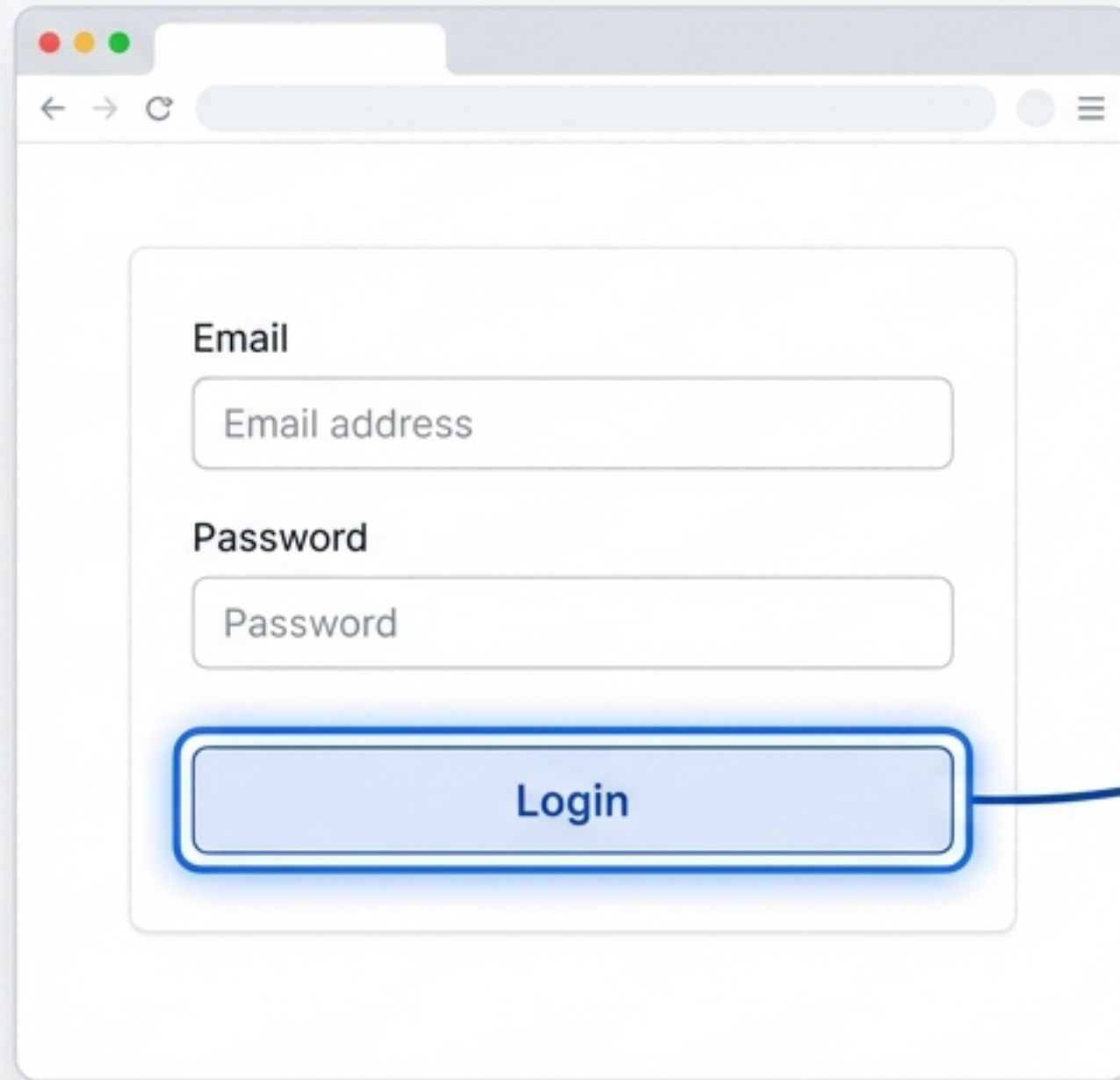


5

USE (Copy & Paste)

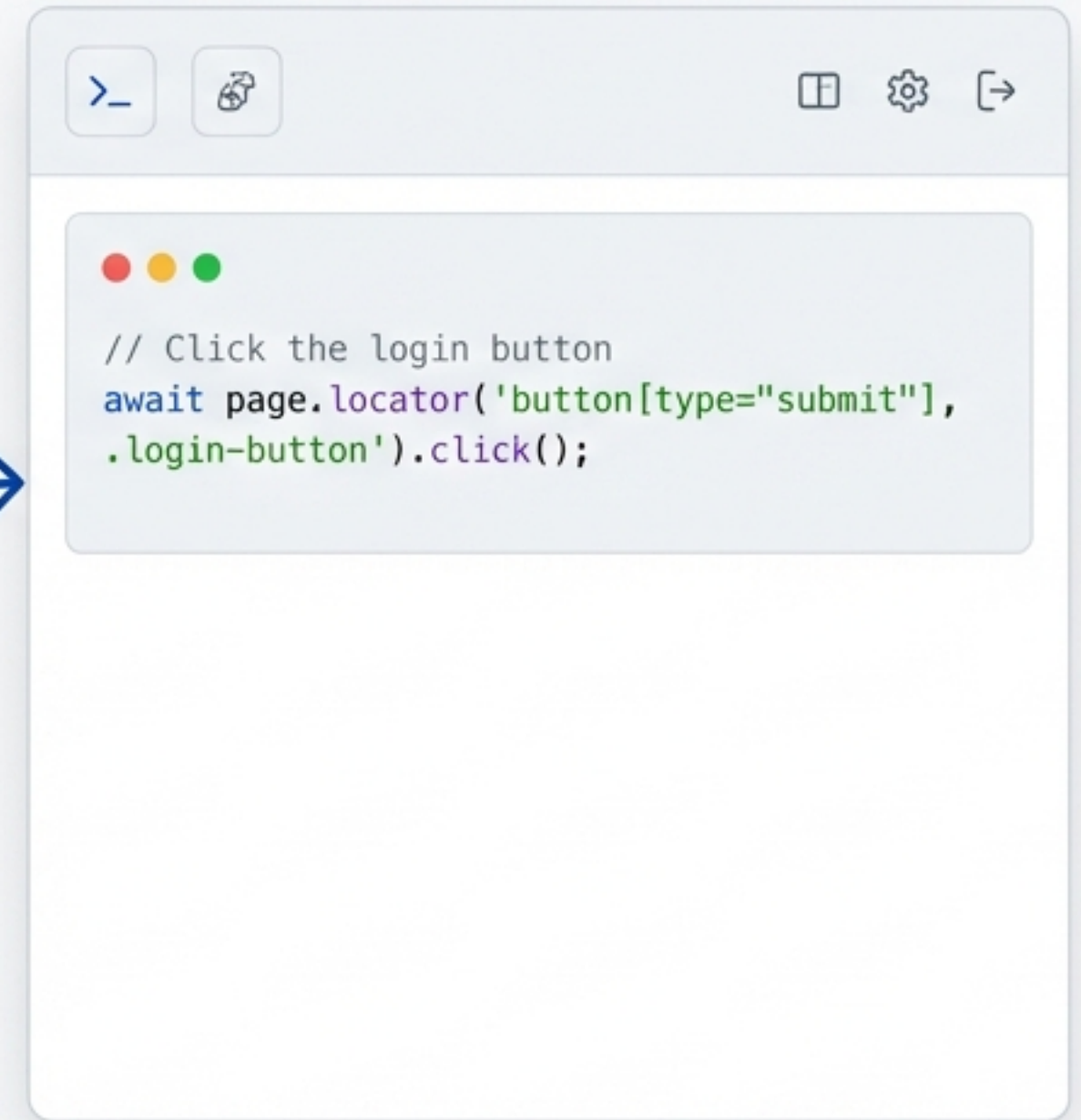
Click 'Copy Code' and paste it directly into your automation framework.

From Web Element to Test Code in Seconds



A web browser window displaying a login form. The form has two input fields: "Email" with placeholder text "Email address" and "Password" with placeholder text "Password". Below these fields is a blue "Login" button. The browser's address bar and navigation icons are visible at the top.

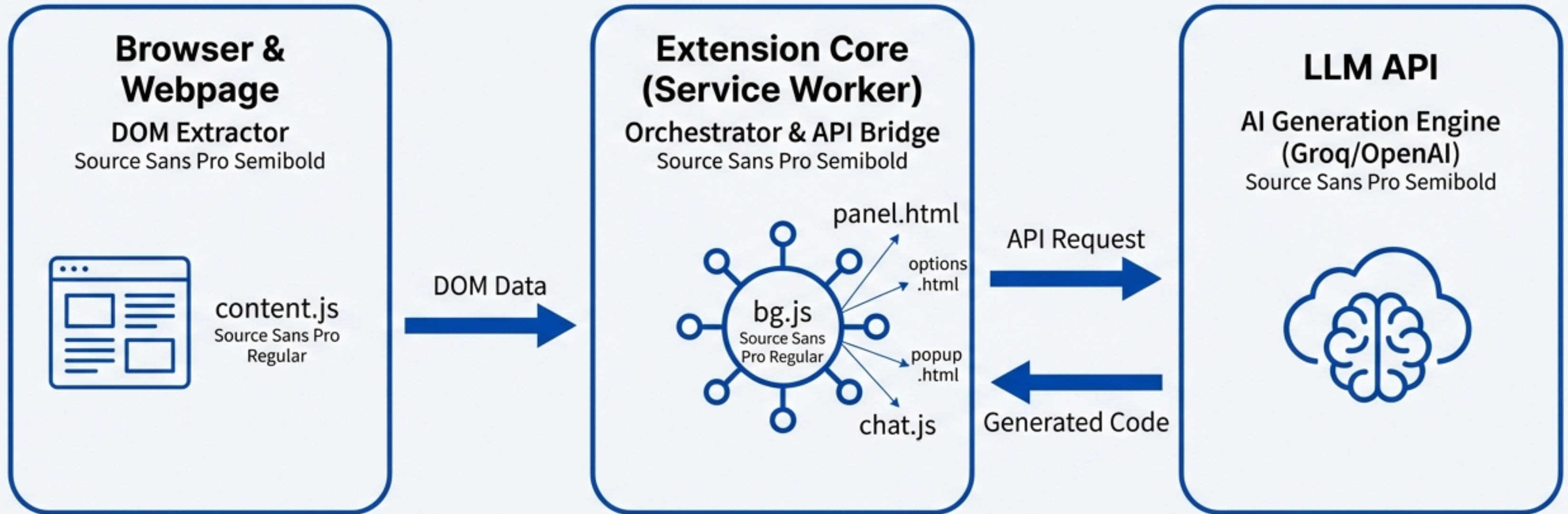
Generated in ~3 seconds



```
// Click the login button
await page.locator('button[type="submit"],
.login-button').click();
```

A code editor window showing the generated test code. The code is written in a light blue box with a header bar containing icons for running, saving, and settings. The code is a single line of JavaScript that uses Playwright's locator to find the login button and click it.

System Architecture at a Glance



Deconstructing the Extension: Core Components

Configuration & Manifest



manifest.json

The extension's blueprint. Defines permissions, configurations, and entry points (`background.js`, `content.js`, `content.js`). (Analogy: 'Similar to pom.xml').

User Interface (UI)



panel.html

The HTML structure for the extension's side panel.



popup.js

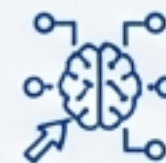
Manages API key input and storage.



tabs.js

Handles tab navigation within the panel UI.

Core Logic & Communication



content.js

Injected into webpages to let users select elements and extract their DOM.



bg.js (Service Worker)

The central bridge. Receives DOM from `content.js`, calls the LLM API, and sends generated code to the UI.



chat.js

Receives and displays the generated code in the UI for the user.

The Brain of the Operation: The LLM Engine

`Prompt.js` - The Instruction Set

Contains the instruction templates that guide the AI. It tells the LLM **what** to do with the DOM data.

```
"Generate a {framework} test script for this HTML element: {dom_element}"
```

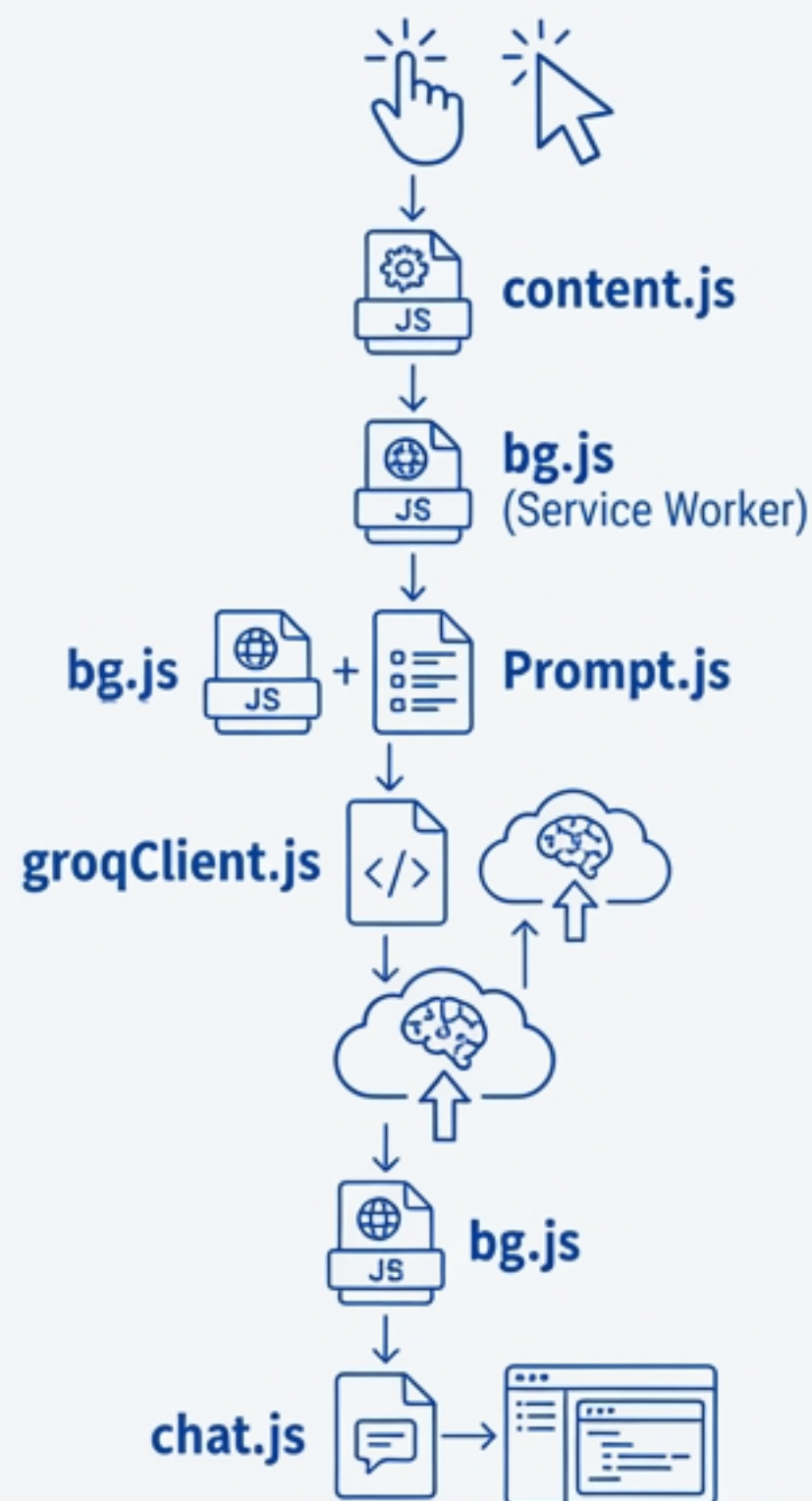


`groqClient.js` / `openAIClient.js` - The Messenger

The dedicated client that connects to the LLM API service (Groq or OpenAI), sends the combined prompt and DOM, and handles the response.



Tracing the Flow: From Click to Generated Code



1. User clicks "Inspect" then clicks an element on a webpage.

2. Captures the element's DOM.

3. Receives the DOM data from `content.js`.

4. Combines the DOM with the correct instruction template.

5. Sends the final payload to the external LLM API.

6. Processes the request and returns the generated test code.

7. Receives the code from the API.

8. Receives the code from `bg.js` and renders it in the extension's UI panel.

The Critical Bridge: `content.js` and `bg.js`

`content.js` (The Field Agent)



Role:

Injected directly into the webpage.

Responsibilities:

- Listens for user clicks after "Inspect" is activated.
- Extracts the selected element's HTML/DOM.
- Sends this isolated data to `bg.js`.

`bg.js` (The Command Center)



Role:

Runs as a persistent background service worker.

Responsibilities:

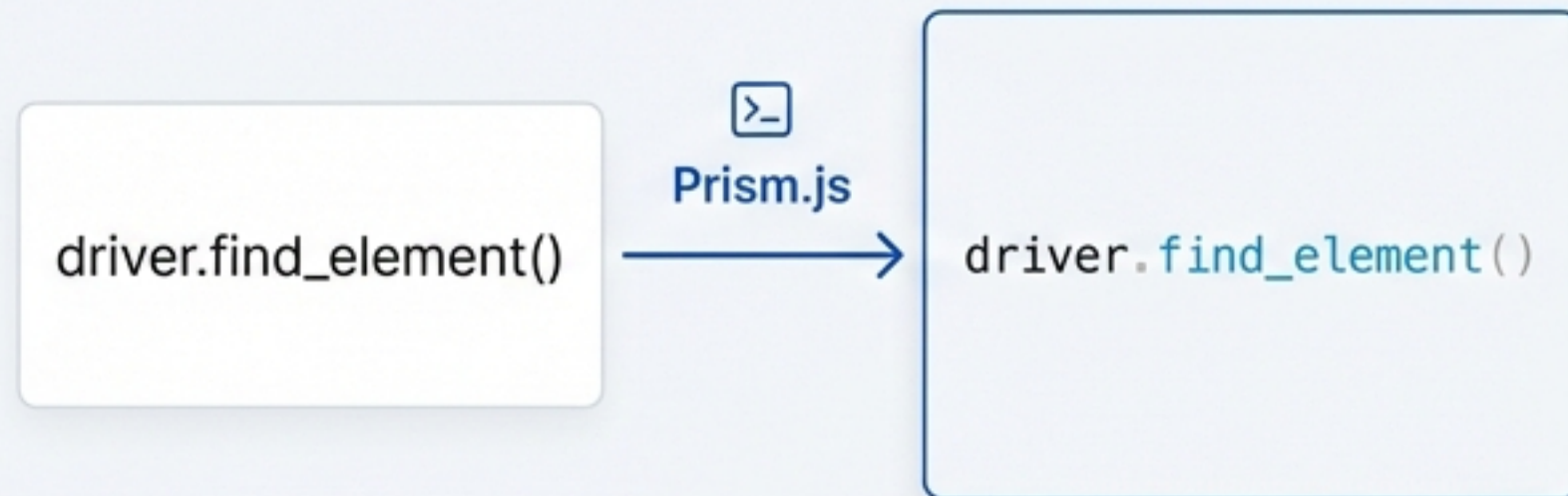
- Receives data from any number of content scripts.
- Manages all external API calls to the LLM.
- Routes the final generated code back to the correct UI panel.

Beyond Function: A Polished Developer Experience

The final output is not just raw text, but clean, readable, and aesthetically pleasing code.

Prism.js: For Code Readability

Makes code colorful & pretty.



Marked.js: For Clear Formatting

Converts markdown to readable text.

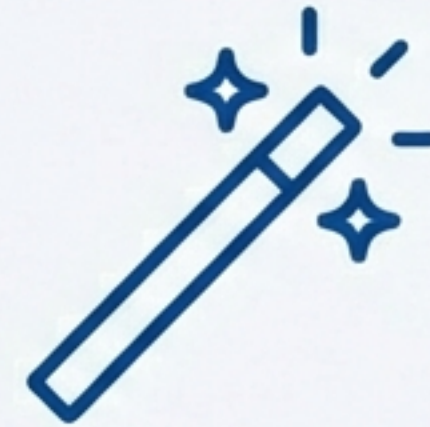


Technology Stack Summary



Frontend

HTML, CSS, JavaScript



Code Presentation

Prism.js, Marked.js



Extension Framework

Chrome Extension APIs
(Service Worker, Content
Scripts)



AI Engine

LLM API (Groq / OpenAI)

Stop Writing Boilerplate. Start Accelerating Your Tests.



This AI extension transforms a **bottleneck** in the testing lifecycle into an opportunity for **speed and efficiency**. It's not just about generating code; it's about **freeing up developer time** to focus on building **better** software.